LS Lab 5: Using Large Language Models (LLMs) in DevOps Pipelines

In this lab, students explore how Large Language Models (LLMs) can be used as assistive tools in DevOps environments. The lab focuses on integrating an LLM into an existing CI/CD pipeline with Kubernetes deployment and security checks, and evaluating how LLMs can support explanation, troubleshooting, and security-aware decision-making.

Prerequisites

- Virtualization
- Containerization (Docker)
- CI/CD pipelines
- Basic Linux and Git knowledge

Students will reuse:

- The CI/CD pipeline implemented in **Lab 3**
- A containerized application from previous labs
- A Kubernetes deployment configuration

Task 1: CI/CD Pipeline Baseline with Security and Kubernetes

Before introducing the LLM, you will prepare a **secure DevOps baseline**.

1- Show the CI/CD pipeline you implemented in Lab3, including the following stages:

- Source code checkout
- Build
- Test
- Container image build and deploy

2- Add a Static Application Security Testing (SAST) stage to your pipeline (using tools like Bandit, Semgrep, Sonarqube). Show the output of the security scan

3- Extend your pipeline to include a deployment stage to Kubernetes.

Task 2: LLM Setup and Deployment

In this task, you will prepare an LLM that will later be used in the CI/CD pipeline.

1- Choose an external LLM API (open AI chat API, Gemini API, YAndex GPT API), show how you obtained an API key and tested it locally using a simple prompt.

2- Add a new stage called ***llm-review*** to your existing CI/CD pipeline that will send application source code or pipeline configuration to the LLM and store the response as a pipeline artifact. Then Show the updated pipeline stage.

3- Why should the LLM stage be placed after the build and test stages ?

Task 3: Testing and Validation

1- Trigger the pipeline and verify that the LLM stage runs successfully. Provide evidence from the pipeline logs and show the result of the artifact.

2- Does the LLM output correctly describe or explain the application or the pipeline? Briefly explain your observation.

3- How does the LLM output help a developer during CI/CD execution?

Task 4: Security and Trust Analysis

1- what security risks can arise when sending source code, logs or configurations to an LLM?

2a- Modify your current LLM prompt so that the model analyzes the CI/CD pipeline from a security, reliability, and trust perspective. The prompt should explicitly ask the LLM to identify potential risks, vulnerabilities, and failure points, and to justify its observations.

b- Run the refined prompt, report the LLM's output, and critically assess which points you consider valid, questionable, or incorrect

Bonus Task: Advanced Use of Large Language Models in DevOps

1- Deploy a lightweight LLM model (phi-3, llama3, mistral-quantized). Show the commands used to install the runtime and start the model.

Hint: *Hugging Face, for AI you can use llama3*

2- Extend the CI/CD pipeline so the LLM performs advanced analysis. The LLm should be able to:

- Analyze logs from a failed CI/CD job and suggest possible fixes.
- Detects outdated dependencies or code quality issues.
- Provide concise improvement suggestions for documentation on the README file.

3- Design a workflow where the LLM produces a summary of detected issues or improvements and creates a pull (merge) request.